

Lidar Guarding

For instructions on app connection, please consult the tutorial located in “1.Getting Ready/1.4 App Control”.

1. Enable Game

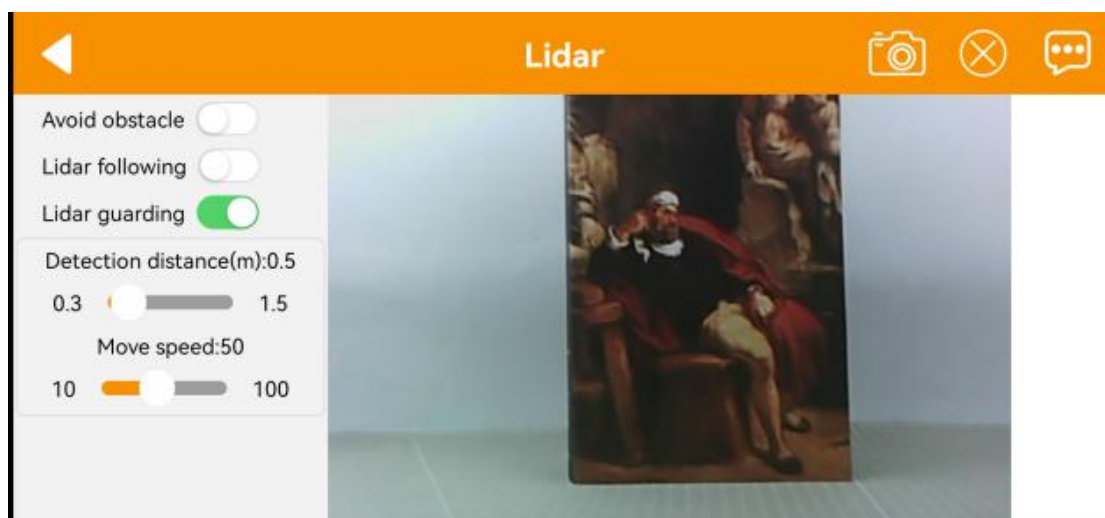
Note: The MentorPi of Ackerman version does not support Lidar guarding function.

1.1 Initiate Lidar Game through App

- 1) Connect the robot to “WondePi” app.
- 2) Click-on Lidar to navigate to the game interface.



- 3) Switch on “Lidar guarding” button to start the game.



1.2 Initiate Lidar Game Using Command

1) Start the robot, and access the robot system desktop using VNC according to the tutorial saved in "8.Set Development Environment/Lesson 1 VNC Installation and Connection".



2) Click-on  to open the command line terminal.

3) Execute the command and press Enter to deactivate the automatic start service for the app.

~/./stop_ros.sh

```
> ~/./stop_ros.sh
zsh: killed ~/./stop_ros.sh
```

4) Run the command to enable the local services for app-related game and chassis control services.

ros2 launch app lidar_node.launch.py debug:=true

```
> ~/./stop_ros.sh
zsh: killed ~/./stop_ros.sh
[?] | ~ ▶ ◀ KILL x | 09:53:18
ros2 launch app lidar_node.launch.py debug:=true
```

5) Open a new terminal, and execute the command. Then, press Enter to start the Lidar game.

ros2 service call /lidar_app/enter std_srvs/srv/Trigger {}

```
> ros2 service call /lidar_app/enter std_srvs/srv/Trigger {}
waiting for service to become available...
requester: making request: std_srvs.srv.Trigger_Request()

response:
std_srvs.srv.Trigger_Response(success=True, message='enter')
```

Note: The robot's performance initiated by the app and command is identical.

6) Enter the command and hit Enter to launch Lidar guarding game.

ros2 service call /lidar_app/set_running interfaces/srv/SetInt64 "{data: 3}"

```
> ros2 service call /lidar_app/set_running interfaces/srv/SetInt64 "{data: 3}"
waiting for service to become available...
requester: making request: interfaces.srv.SetInt64_Request(data=3)

response:
interfaces.srv.SetInt64_Response(success=True, message='set_running')
```

7) If you want to exit the game, enter the command and press “Enter”.

```
ros2 service call /lidar_app/set_running interfaces/srv/SetInt64 "{data: 0}"
```

```
> ros2 service call /lidar_app/set_running interfaces/srv/SetInt64 "{data: 0}"
requester: making request: interfaces.srv.SetInt64_Request(data=0)

response:
interfaces.srv.SetInt64_Response(success=True, message='set_running')
```

Note: The game will continue to run under the current Raspberry Pi power-on state if not exited. To avoid excessive use of the Raspberry Pi's operating memory, please follow the above instructions to close the current game before executing other games.

8) If you need to terminate the program, press short-cut “Ctrl+C” on the terminal opened in steps 4) and 5).

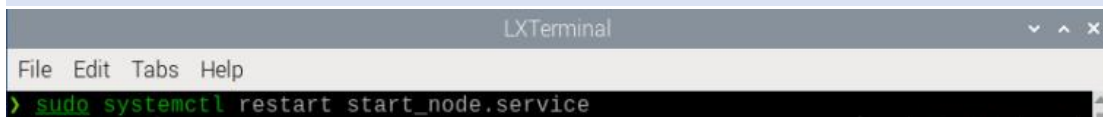
After experiencing the Lidar game, you can activate the app service either by using a command or restarting the robot. If the app service is not activated, related app functions will be disabled. In the case of a robot restart, the app service will start automatically.



Click  and enter the command. Press enter to start the app, and wait for the buzzer to beep.

Note: please enter the command in the system path, not in the Docker container.

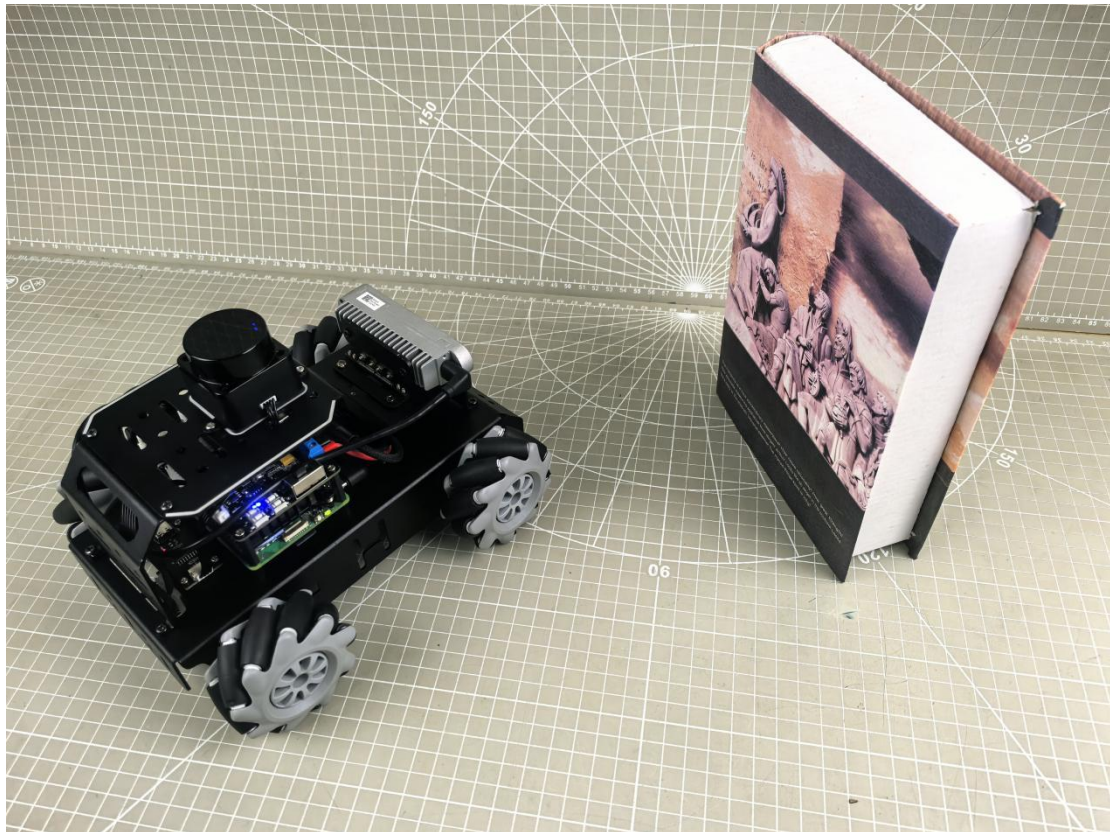
```
sudo systemctl restart start_node.service
```

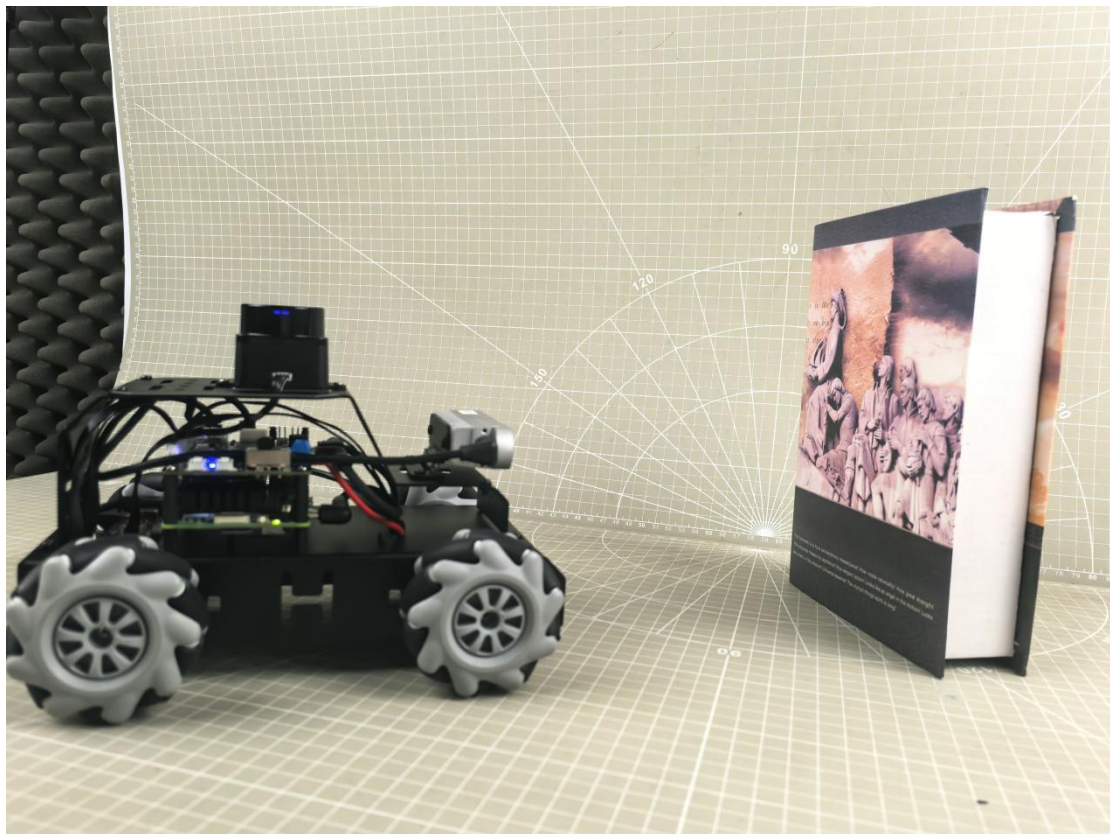


2. Program Outcome

Let's use a book as the object to be detected. When using the Lidar following

function, it's important to ensure that the object to be detected is higher than the scanning height of the Lidar mounted on the MentorPi. This will enable the Lidar to effectively scan the position information of the object. Once the position information is obtained, the MentorPi will adjust its orientation to face towards the obstacle. It means that the camera will face directly towards the obstacle.



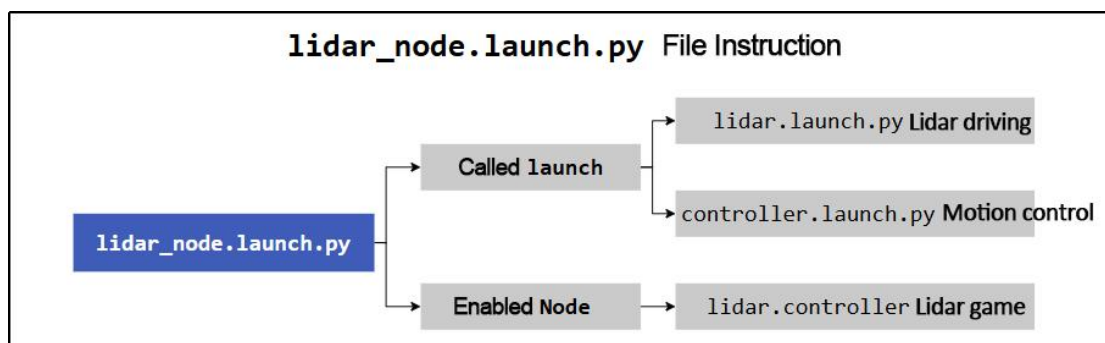


3. Program Analysis

3.1 Launch File

The launch file is located in:

/home/ubuntu/ros2_ws/src/app/launch/lidar_node.launch.py



```

1 import os
2
3 from launch_ros.actions import Node
4 from launch.conditions import IfCondition
5 from launch import LaunchDescription, LaunchService
6 from launch.substitutions import LaunchConfiguration
7 from launch.launch_description_sources import PythonLaunchDescriptionSource
8 from launch.actions import IncludeLaunchDescription, OpaqueFunction, GroupAction, DeclareLaunchArgument
9
10 def launch_setup(context):
11     compiled = os.environ['need_compile']
12     debug = LaunchConfiguration('debug', default='false')
13     debug_arg = DeclareLaunchArgument('debug', default_value=debug)
14     if compiled == 'True':
15         controller_package_path = get_package_share_directory('controller')
16         peripherals_package_path = get_package_share_directory('peripherals')
17     else:
18         controller_package_path = '/home/ubuntu/ros2_ws/src/driver/controller'
19         peripherals_package_path = '/home/ubuntu/ros2_ws/src/peripherals'
20
21     lidar_controller_node = GroupAction([
22         IncludeLaunchDescription(
23             PythonLaunchDescriptionSource(
24                 os.path.join(peripherals_package_path, 'launch/lidar.launch.py')),
25                 condition=IfCondition(debug),
26             ),
27         IncludeLaunchDescription(
28             PythonLaunchDescriptionSource(
29                 os.path.join(controller_package_path, 'launch/controller.launch.py')),
30                 condition=IfCondition(debug),
31             ),
32         Node(
33             package='app',
34             executable='lidar_controller',
35             output='screen',
36         ),
37     ])
38 
```

◆ Set the storage path

Retrieve the paths for the two packages: peripherals and controller.

```

14     if compiled == 'True':
15         controller_package_path = get_package_share_directory('controller')
16         peripherals_package_path = get_package_share_directory('peripherals')
17     else:
18         controller_package_path = '/home/ubuntu/ros2_ws/src/driver/controller'
19         peripherals_package_path = '/home/ubuntu/ros2_ws/src/peripherals'

```

◆ Initiate other Launch files

```

21     lidar_controller_node = GroupAction([
22         IncludeLaunchDescription(
23             PythonLaunchDescriptionSource(
24                 os.path.join(peripherals_package_path, 'launch/lidar.launch.py')),
25                 condition=IfCondition(debug),
26             ),
27         IncludeLaunchDescription(
28             PythonLaunchDescriptionSource(
29                 os.path.join(controller_package_path, 'launch/controller.launch.py')),
30                 condition=IfCondition(debug),
31             ),
32     ])

```

lidar.launch.py Lidar launch

controller.launch.py Motion control launch

◆ Initiate Node

```

34         Node(
35             package='app',
36             executable='lidar_controller',
37             output='screen',
38             ),
39     ])

```

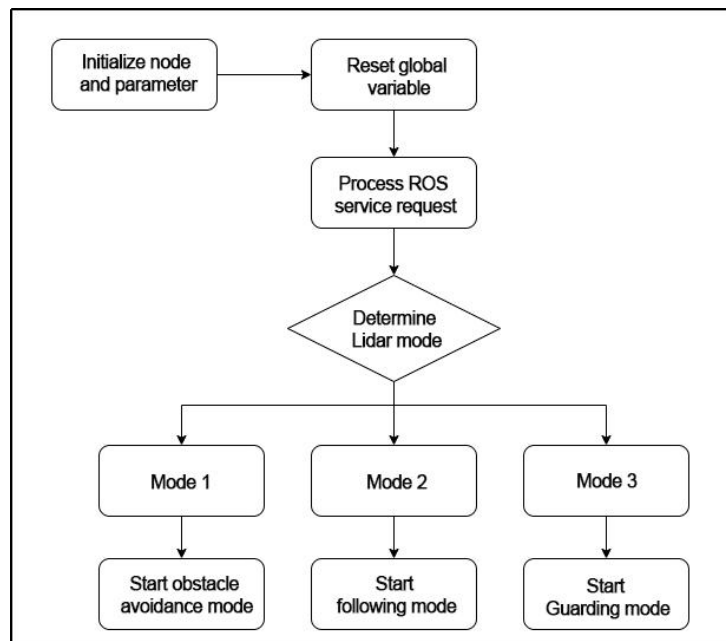
Enable Lidar game node.

3.2 Source Code File

The source code of the program is saved in:

/home/ubuntu/ros2_ws/src/app/app/lidar_controller.py

The process logic of the program based on the implemented effect is shown in the diagram:



◆ Initialization

```

25 class LidarController(Node):
26     def __init__(self, name):
27         rclpy.init()
28         super().__init__(name)
29
30         self.name = name
31         self.running_mode = 0
32         self.threshold = 0.6 # meters
33         self.scan_angle = math.radians(360) # radians
34         self.speed = 0.2
35         self.last_act = 0
36         self.timestamp = 0
37         self.angle_data = []

```

self.name: stores the incoming name as the property of the object.

Other attributes include parameters for controlling the Lidar, such as operating

mode, threshold, scanning angle, and speed.

self.lock: creates a thread lock for safe access in a multi-threaded environment.

◆ Create ROS service

```

46 self.create_service(Trigger, '/enter', self.enter_srv_callback) # 进入游戏(enter the game)
47 self.create_service(Trigger, '/exit', self.exit_srv_callback) # 退出游戏(exit the game)
48 self.create_service(SetInt64, '/set_running', self.set_running_srv_callback) # 开启游戏(start the game)
49 Heart(self, self.name + '/heartbeat', 5, lambda : self.exit_srv_callback(request=Trigger.Request(), response=Trigger.Response())) # 心跳包(heartbeat package)
50 self.create_service(SetFloat64List, '/set_param', self.set_parameters_srv_callback) # 参数设置(set parameter)

```

Enter game: A ROS service named `"/enter"` is created with the type `Trigger`, and the callback function is `enter_srv_callback`. When this service is called, it will execute the `enter_srv_callback` function.

Exit game: A ROS service named `"/exit"` is created with the type `Trigger`, and the callback function is `exit_srv_callback`. When this service is called, it will execute the `exit_srv_callback` function.

Enable game: A ROS service named `"/set_running"` is created with the type `SetInt64`, and the callback function is `set_running_srv_callback`. When this service is called, it will execute the `set_running_srv_callback` function.

Set parameters: A ROS service named `"/set_param"` is created with the type `SetFloat64List`, and the callback function is `set_parameters_srv_callback`.

When this service is called, it will execute the `set_parameters_srv_callback` function.

◆ Lidar guarding

```

245 elif self.running_mode == 3:
246     # 拼接距离数据, 从右半侧逆时针到左半侧(the merged distance data from right half counterclockwise to the left half)
247     ranges = np.append(right_range[:-1], left_range)
248     nonzero = ranges.nonzero()
249     nonan = np.isfinite(ranges[nonzero])
250     dist = ranges[nonzero][nonan]
251     if len(dist) > 1:
252         dist = dist.min()
253         min_index = list(ranges).index(dist)
254
255         angle = -angle + lidar_data.angle_increment * min_index # 计算最小值对应的角度(calculate the angle corresponding to the minimum value)
256         if dist < self.threshold and abs(math.degrees(angle)) > 5: # 控制左右(control the left and right)
257             if self.lidar_type != 'G4':
258                 self.pid_yaw.update(-angle)
259                 twist.angular.z = common.set_range(self.pid_yaw.output, -self.speed * 6.0, self.speed * 6.0)
260             elif self.lidar_type == 'G4':
261                 self.pid_yaw.update(angle)
262                 twist.angular.z = -common.set_range(self.pid_yaw.output, -self.speed * 6.0, self.speed * 6.0)
263             else:
264                 self.pid_yaw.clear()
265                 if abs(twist.angular.z) < 0.008:
266                     twist.angular.z = 0.0
267                 self.mecanum_pub.publish(twist)

```

The code concatenates the distance data from the left and right sides of the robot, obtained from the Lidar, into an array `ranges` to form a circular scan. It then calculates the minimum distance value `dist` and its corresponding angle `angle` from the `ranges` array.

If dist is less than a threshold value and the angle deviates from the forward direction by more than 5 degrees, the Yaw axis PID controller will be activated. Based on the output of the PID controller, the code calculates the turning angular velocity `twist.angular.z`. The control commands are published as a Twist message to the robot to execute the desired actions.